UNIT 3

UNIT - III Semi-Supervised Learning & Text Feature Engineering
Introduction, understanding semi-supervised learning, Semi-supervised algorithms in action, Self-training, implementing self-training, Finessing your self-training implementation, Contrastive Pessimistic Likelihood Estimation
Text Feature Engineering: Introduction, Text feature engineering, cleaning text data, Text cleaning with Beautiful Soup, managing punctuation and tokenizing, Tagging and categorizing words, creating features from text data, stemming, Bagging and random forests, Testing our prepared data

Understanding Semi-Supervised Learning

One of the biggest challenges in machine learning is obtaining labeled data for training. Since machine learning models require labeled data to learn effectively, but labeling data manually is expensive, time-consuming, and sometimes prone to errors (especially in complex domains like medical diagnosis), semi-supervised learning (SSL) offers a way to overcome this limitation.

What is Semi-Supervised Learning?

Semi-supervised learning is a type of machine learning that utilizes both **labeled** and **unlabeled** data to improve learning accuracy. It lies between **supervised learning** (which uses only labeled data) and **unsupervised learning** (which uses only unlabeled data).

- **Supervised Learning** → Uses labeled data for training. Example: Classifying emails as spam or not spam.
- **Unsupervised Learning** → Uses only unlabeled data to identify patterns. Example: Clustering customers based on purchasing behaviour.
- **Semi-Supervised Learning** → Uses a small amount of labeled data and a large amount of unlabeled data. Example: Image classification with only a few manually labeled images.

By leveraging both labeled and unlabeled data, semi-supervised techniques can achieve better performance than purely supervised or unsupervised approaches.

Types of Techniques that lies b/w supervised and unsupervised learning:

1. **Semi-Supervised Learning Techniques**
   - These techniques use both labeled and unlabeled data to improve classification models.
   - A portion of the data is set aside as test data for model evaluation.
2. **Transductive Learning Techniques**
   - These focus purely on labeling the **unlabeled** data using the available labeled data.
   - Unlike semi-supervised learning, transductive learning may not involve a test dataset.
3. **Active Learning Techniques**
   - These techniques allow the model to **select the most useful examples** from the dataset to be labeled manually.
   - This approach reduces the amount of labeled data required by focusing on data points that improve model performance the most.

Semi-supervised algorithms in action

- Now that we understand **what semi-supervised learning is** and **why it is useful**, it's time to move beyond theory and explore how to **implement these techniques** in real-world applications.
- In the next sections, we will shift from general explanations to **practical implementations** of semi-supervised algorithms. By applying these techniques effectively, we can enhance model performance even when **only a small portion of the dataset is labeled**.

Self-Training in Semi-Supervised Learning

- Now that we understand the concept of **semi-supervised learning**. One of the simplest and most commonly used semi-supervised techniques is **self-training**. This method is widely used in fields like **Natural Language Processing (NLP)** and **computer vision** due to its efficiency. However, while self-training offers significant advantages, it also presents potential risks.

How Self-Training Works

- Self-training iteratively assigns labels to unlabeled data by leveraging the information from labeled examples. The process follows these steps:
    1. A model is trained using labeled data and then predicts labels for **unlabeled data**.
    2. The model assigns a **confidence score** to each prediction.
    3. Only the most **confident predictions** (above a set threshold) are added to the labeled dataset.
    4. The model is retrained with this **expanded labeled dataset**.
    5. Steps **1 to 4 are repeated** until the model successfully converges.
- This method is often implemented as a **wrapper around a base model**. In this chapter, we will use **Support Vector Machines (SVMs)** as the base model for self-training.

Example: semi-supervised learning in a university student performance prediction system.

Scenario: Predicting Student Performance

- Imagine you are developing a **machine learning model** to predict whether a student will **pass or fail** based on their **attendance, study hours, and assignment scores**.

Available Data:

- **Labeled Data**: You have **100 students** for whom you know the final result (Pass or Fail).
- **Unlabeled Data**: You have **900 students** for whom you only have attendance, study hours, and assignment scores, but their final result is unknown.

Steps of Semi-Supervised Learning

Step 1: Train a Model on Labeled Data

- You train a **classification model** (e.g., Logistic Regression, Decision Tree) using the 100 labeled students.
- The model learns patterns like:
    - Students with **low attendance (< 50%)** and **low study hours (< 2 hrs/day)** often **fail**.
    - Students with **high attendance (> 80%)** and **high study hours (> 4 hrs/day)** often **pass**.

Step 2: Predict Labels for Unlabeled Data

- You use the trained model to predict **whether the remaining 900 students will pass or fail**.
- Each prediction comes with a **confidence score** (probability).

Step 3: Select High-Confidence Predictions

- If the model predicts **"Pass" with 95% confidence**, we assume it is **correct**.
- If the model predicts **"Fail" with 90% confidence**, we assume it is **correct**.
- If the confidence is **low (e.g., 60%)**, we do **not** use the prediction.

Step 4: Add High-Confidence Predictions to the Labeled Dataset

- Let's say **500 students** had high-confidence predictions.
- We **add them to our labeled dataset**.

Step 5: Retrain the Model

- Now, we train the model again using:
    - **100 original labeled students**
    - **+ 500 newly labeled students** (high-confidence predictions)
    - **= 600 labeled students** in total

Step 6: Repeat Until No More Confident Labels Can Be Added

- The model is now **more accurate** because it was trained on more data.
- We repeat Steps **2 to 5** until no more high-confidence predictions remain.

Final Result

- The model now has **higher accuracy** compared to using only the initial 100 labeled students.
- We successfully labeled most of the 900 students **without manual effort**.

Advantages & Risks of Self-Training

Advantages:

- **Saves time** by reducing manual labeling effort.
- **Expands training data** dynamically, improving model performance.

- Can be **easily integrated** with existing classifiers like SVMs.

Risks:

- **Incorrect predictions** in early iterations may lead to error propagation.
- **Overfitting** to the self-labeled data, especially if the confidence threshold is not well-calibrated.

Implementing self-training

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

- **NumPy (np)**: For numerical operations
- **Pandas (pd)**: For handling dataset
- **RandomForestClassifier**: Machine learning model to classify student performance
- **accuracy_score**: Measures model accuracy

Step 2: Create the Dataset

```
data = {
   "Attendance": [90, 85, 70, 60, 80, 50, 95, 45, 75, 88, 40, 92, 68, 72, 55, 30, 65, 78, 82, 48],
   "Assignments": [80, 78, 60, 50, 75, 40, 90, 30, 70, 85, 25, 88, 55, 60, 35, 20, 45, 72, 77, 38],
   "Test_Scores": [85, 82, 65, 55, 78, 45, 92, 35, 68, 80, 20, 90, 58, 62, 40, 15, 50, 69, 74, 30],
   "Performance": [1, 1, 1, 0, 1, None, 1, 0, 1, 1, 0, 1, None, None, 0, 0, 0, 1, 1, None]
}
df = pd.DataFrame(data)
```

- Creates a dataset where:
  - **Attendance, Assignments, and Test Scores** are student features.
  - **Performance** is the label (1 = Pass, 0 = Fail).
  - Some labels are **missing (None)**, meaning we have **unlabeled data**.

Step 3: Separate Labeled & Unlabeled Data

```
labeled_data = df.dropna(subset=["Performance"])
unlabeled_data = df[df["Performance"].isna()]
```

- **Labeled Data** → Contains students with known performance (1 or 0).
- **Unlabeled Data** → Contains students with unknown performance (None).

Step 4: Define Features (X) & Labels (y)

```
X_labeled = labeled_data.drop(columns=["Performance"])
y_labeled = labeled_data["Performance"].astype(int)  # Convert to integer
X_unlabeled = unlabeled_data.drop(columns=["Performance"])
```

- **X_labeled** → Features (Attendance, Assignments, Test Scores) for labeled students.
- **y_labeled** → Labels (1 = Pass, 0 = Fail) for labeled students.
- **X_unlabeled** → Features of students with unknown performance.

Step 5: Train Initial Model

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_labeled, y_labeled)
```

- Uses a **Random Forest Classifier** with **100 decision trees** to learn from labeled data.
- The model is trained on X_labeled and y_labeled.

Step 6: Define Iterative Self-Learning Process

```
max_iter = 10  # Maximum iterations
confidence_threshold = 0.9  # Confidence threshold
```

- **max_iter = 10** → The model will re-train up to **10 times** with new pseudo-labeled data.
- **confidence_threshold = 0.9** → The model only adds pseudo-labels if it's **90% confident**.

Step 7: Iterate & Add High-Confidence Pseudo-Labels

```
for iteration in range(max_iter):
   pseudo_probs = model.predict_proba(X_unlabeled)
   pseudo_labels = np.argmax(pseudo_probs, axis=1)
   confidence_scores = np.max(pseudo_probs, axis=1)
```

- Predicts **probabilities** for each unlabeled student.
- Extracts **pseudo-labels** (1 or 0) with the highest probability.
- Gets **confidence scores** (probability of the predicted class).

Step 8: Select High-Confidence Predictions

```
high_confidence_idx = np.where(confidence_scores >= confidence_threshold)[0]

if len(high_confidence_idx) == 0:
    print(f"Iteration {iteration + 1}: No high-confidence samples found. Stopping early.")
    break
```

- Finds **indexes** where confidence is **≥ 90%**.
- If no samples are high-confidence, **stop training** early.

Step 9: Extend Training Data with Pseudo-Labels

```
X_train_extended = np.vstack([X_labeled, X_unlabeled.iloc[high_confidence_idx]])
y_train_extended = np.concatenate([y_labeled, pseudo_labels[high_confidence_idx]])
```

- Adds the high-confidence **pseudo-labeled** students to the training data.

Step 10: Retrain the Model

```
model.fit(X_train_extended, y_train_extended)
print(f"Iteration {iteration + 1}: Added {len(high_confidence_idx)} new pseudo-labeled samples.")
```

- The model **trains again** with the extended dataset.
- Prints how many **new pseudo-labeled** students were added.

Step 11: Remove Used Samples from Unlabeled Data

```
X_unlabeled = X_unlabeled.drop(X_unlabeled.index[high_confidence_idx])

if X_unlabeled.empty:
    print("All samples are now labeled. Stopping.")
    break
```

- **Removes** high-confidence pseudo-labeled samples from the unlabeled dataset.
- If no students are left without labels, **stop training**.

Step 12: Evaluate Final Model

```
y_pred_final = model.predict(X_labeled)
print("Final Model Accuracy on Labeled Data:", accuracy_score(y_labeled, y_pred_final))
```

- **Tests the final model** on labeled data.
- Prints **Final Accuracy** after using pseudo-labels.

Expected Output

```
Iteration 1: Added 3 new pseudo-labeled samples.
Iteration 2: Added 2 new pseudo-labeled samples.
Iteration 3: No high-confidence samples found. Stopping early.
Final Model Accuracy on Labeled Data: 94%
```

Full Code:

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Sample dataset: [Attendance, Assignments, Test_Scores] → Performance (Pass=1, Fail=0)
data = {
    "Attendance": [90, 85, 70, 60, 80, 50, 95, 45, 75, 88, 40, 92, 68, 72, 55, 30, 65, 78, 82, 48],
    "Assignments": [80, 78, 60, 50, 75, 40, 90, 30, 70, 85, 25, 88, 55, 60, 35, 20, 45, 72, 77, 38],
    "Test_Scores": [85, 82, 65, 55, 78, 45, 92, 35, 68, 80, 20, 90, 58, 62, 40, 15, 50, 69, 74, 30],
    "Performance": [1, 1, 1, 0, 1, None, 1, 0, 1, 1, 0, 1, None, None, 0, 0, 0, 1, 1, None]  # 0 = Fail, 1 = Pass
}

# Convert dataset to DataFrame
```

```
df = pd.DataFrame(data)

# Separate labeled and unlabeled data
labeled_data = df.dropna(subset=["Performance"])
unlabeled_data = df[df["Performance"].isna()]

# Features and Labels
X_labeled = labeled_data.drop(columns=["Performance"])
y_labeled = labeled_data["Performance"].astype(int)  # Convert to integer
X_unlabeled = unlabeled_data.drop(columns=["Performance"])

# Train the initial model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_labeled, y_labeled)

# Iterative Self-Learning Process
max_iter = 10  # Maximum iterations
confidence_threshold = 0.9  # Confidence threshold

for iteration in range(max_iter):
    # Predict probabilities for unlabeled data
    pseudo_probs = model.predict_proba(X_unlabeled)
    pseudo_labels = np.argmax(pseudo_probs, axis=1)  # Get predicted class
    confidence_scores = np.max(pseudo_probs, axis=1)  # Get confidence scores

    # Select high-confidence predictions
    high_confidence_idx = np.where(confidence_scores >= confidence_threshold)[0]

    if len(high_confidence_idx) == 0:
        print(f"Iteration {iteration + 1}: No high-confidence samples found. Stopping early.")
        break  # Stop if no more high-confidence samples

    # Add high-confidence pseudo-labels to training data
    X_train_extended = np.vstack([X_labeled, X_unlabeled.iloc[high_confidence_idx]])
    y_train_extended = np.concatenate([y_labeled, pseudo_labels[high_confidence_idx]])

    # Retrain the model
    model.fit(X_train_extended, y_train_extended)

    print(f"Iteration {iteration + 1}: Added {len(high_confidence_idx)} new pseudo-labeled
samples.")

    # Remove used samples from unlabeled data
    X_unlabeled = X_unlabeled.drop(X_unlabeled.index[high_confidence_idx])

    # Stop if no more unlabeled data
    if X_unlabeled.empty:
        print("All samples are now labeled. Stopping.")
        break

# Final Evaluation on the labeled test set
y_pred_final = model.predict(X_labeled)
print("Final Model Accuracy on Labeled Data:", accuracy_score(y_labeled, y_pred_final))
```

Finessing your self-training implementation

In this section, we revisit self-training algorithms and discuss some potential issues we observed in earlier trials, such as questionable accuracy and variance. Self-training can be fragile; if the algorithm is misconfigured or if the data has peculiarities, errors can accumulate, making future iterations worse.

Risks in Self-Training

1. **Unhelpful Labeled Data**: Initially, adding labeled data might not improve the model. The cases that are easiest to label are often similar to existing ones, but adding them might not help future labeling or could misguide the classifier.
2. **Bias in Selection**: If the labeled data is biased (favoring one class over others), the model may overfit, causing poor generalization. This issue is more likely if the data is not well-balanced.

Diagnosing Issues

- One way to monitor issues is by tracking classification accuracy over early iterations. Visualizing these trends can help identify when the model is misbehaving.

Solutions to Improve Self-Training

1. **Use More Diverse Labeled Data**: Adding more diverse data can help avoid overfitting and kick-start the process. However, keep some data aside for validation to prevent overfitting.
2. **Multiple Model Instances**: Training multiple self-training models on different subsets of the data and running multiple trials can help understand how the data affects performance.
3. **Sampling Techniques**: Consider using multiple sampling techniques in parallel to increase the diversity of the labeled data.
4. **Quality over Quantity**: Instead of increasing the amount of data, improve its diversity. Having a more representative subset of labeled data can lead to better results.

Data Selection and Bias

- **Disproportionate Sampling**: If the dataset is biased toward one class, the self-training algorithm is more likely to overfit, especially when cases similar to pre-existing ones are added.

Validation Techniques

- **Independent Validation**: If possible, hold out a separate validation dataset to test the model's performance.
- **Sample Splitting**: Split the dataset into different parts (training, testing, validation) for more reliable results.
- **Resampling**: Use methods like cross-validation to repeatedly check the model's performance.

Handling Noise in Unlabeled Data

- **Noise in Unlabeled Data**: Introducing unlabeled data often brings noise. Measures like the Fisher's discriminant ratio or error functions of a linear classifier can help assess the separability of data and its impact on performance.

Improving Selection Process

- The key to the self-training algorithm working correctly is the accurate calculation of confidence for each label projection. Confidence calculation is the key to successful self-training.
- Instead of simply selecting all projected labels, you can:
    - Use a **confidence threshold** to pick only the most confident predictions.
    - Add labels and weigh them based on their confidence.

Contrastive Pessimistic Likelihood Estimation

- **Contrastive Pessimistic Likelihood Estimation (CPLE)** is a method in **semi-supervised learning** that addresses the challenges posed by traditional self-training approaches. It aims to balance the strengths of both **supervised** and **semi-supervised** learning, especially when dealing with **expensive labeling** or **high uncertainty in predictions**.
- **Challenges in Semi-Supervised Learning**
    - **Self-training**: In semi-supervised learning, self-training can be a powerful tool, but it has significant risks. The primary risk is that **self-trained classifiers** may end up performing **worse** than fully supervised ones. This is particularly problematic when you have a **large set of unlabeled data** and **expensive labeling costs** (e.g., medical or scientific datasets). In these situations, you cannot afford to have bad predictions, as labeling is costly.

- **Model Degradation**: In some cases, **semi-supervised models** can perform **worse as the dataset size increases**. This happens when the model's predictions on the unlabeled data propagate errors.

- **What Does CPLE Do Differently?**
  - CPLE is a **novel approach** to semi-supervised learning that aims to solve the limitations of self-training classifiers. The core idea is to provide **better predictions** than those generated by equivalent supervised classifiers.
  - CPLE works by explicitly **comparing the performance of the semi-supervised model** to that of a supervised model. In other words, it measures the **relative improvement** of the semi-supervised approach over the supervised one, which leads to more accurate predictions.

How Does CPLE Work?

CPLE consists of five key steps:

- **Step 1: Train a Supervised Model on Labeled Data**
  - The model is first trained using the labeled dataset.
  - This acts as a **baseline model** for comparison.
  - **Example:**
    - We have labeled emails classified as **Spam (S) or Not Spam (NS)**.
    - The model learns to classify emails based on labeled training examples.
- **Step 2: Generate Pseudo-Labels for Unlabeled Data**
  - The model predicts labels for **unlabeled data** using the trained model.
  - These predictions are **not trusted blindly** because errors may exist.
- **Step 3: Compute the Loss Function (Supervised vs. Semi-Supervised Comparison)**
  - The model calculates how much the predictions of the **semi-supervised model** deviate from the **supervised model**.
  - If the semi-supervised model improves performance, the model parameters are **updated accordingly**.
  - If the **supervised model is better**, the model **reduces its reliance** on pseudo-labels.
- **Step 4: Apply Pessimistic Constraints (Avoiding Overconfidence in Pseudo-Labels)**
  - Instead of **blindly trusting pseudo-labels**, CPLE considers all possible labels and picks the one that **minimizes the likelihood gain**.
  - This **prevents error propagation** from incorrect pseudo-labels.
  - **Example of Pessimism in Labeling:**
    - Suppose an email has **80% probability of being Spam (S)** and **20% probability of Not Spam (NS)**.
    - A traditional self-learning model **assigns it as Spam (S)**.
    - **CPLE does not directly assign Spam**; instead, it **evaluates the likelihood of both cases** and chooses the least optimistic outcome to avoid overfitting.
- **Step 5: Model Improvement Over Supervised Learning**
  - By **carefully utilizing unlabeled data**, CPLE outperforms purely supervised models.
  - This is especially useful when labeled data is **scarce**.
- **Real-World Example: Medical or Scientific Data**
  - Imagine you are building a **model to predict a rare disease** based on various diagnostic tests. You might have a small set of labeled data where the disease status is known (e.g., only 10 patients have been diagnosed, and the rest are unlabeled). Labeling new patients would require expensive tests, which is impractical on a large scale.
  - **Supervised model**: A supervised model would use the 10 labeled patients to make predictions on the entire population. While it would work well for those 10, its performance on the large unlabeled dataset would be poor, especially if there are a lot of unknown patterns in the data.

- **CPLE approach**: With CPLE, you can leverage the **unlabeled data** to improve the model, using the known labeled data as a benchmark. The pessimistic assumptions in CPLE help to reduce the risk of overfitting or making overly confident predictions on the unlabeled data.
    - The model will predict labels for the unlabeled data but will remain cautious about those predictions. It will not become overconfident, ensuring better performance on the entire dataset compared to a purely supervised model.

Implementation using Python:

Step-by-Step Explanation

Step 1: Import Libraries

First, we import the required libraries:

```
import numpy as np              # For numerical operations
import random                   # For random sampling
from sklearn.linear_model import SGDClassifier  # Logistic Regression Model
from sklearn.datasets import fetch_openml       # To load datasets
```

Step 2: Load Dataset

The dataset **"10000_songs"** contains features about songs, and the goal is to classify songs into two categories.

```
# Fetch dataset from sklearn
songs = fetch_openml(name="10000_songs", version=1)

# Features (X) and Labels (y)
X = songs.data              # Input Features
ytrue = np.copy(songs.target) # Actual Labels

# Convert -1 labels into 0 (Binary Classification)
ytrue[ytrue == -1] = 0
```

Step 3: Select Labeled and Unlabeled Data

- In semi-supervised learning, **only some data points are labeled**.
- The remaining data points are **unlabeled (-1)**.

```
# Number of labeled samples
labeled_N = 20

# Initialize all data as Unlabeled (-1)
ys = np.array([-1] * len(ytrue))
```

Step 4: Randomly Label Small Amount of Data

Let's label **10 samples as Spam (1)** and **10 samples as Not Spam (0)** randomly.

```
# Randomly select half data as 0 and half as 1
random_labeled_points = random.sample(list(np.where(ytrue == 0)[0]), labeled_N // 2) + \
                        random.sample(list(np.where(ytrue == 1)[0]), labeled_N // 2)

# Assign true labels to these random points
ys[random_labeled_points] = ytrue[random_labeled_points]
```

Step 5: Supervised Learning Model

We train a **Logistic Regression Model** only on the labeled data points.

```
# Base Model using Logistic Regression
basemodel = SGDClassifier(loss='log', penalty='l1')
basemodel.fit(X[random_labeled_points, :], ys[random_labeled_points])
```

```
# Accuracy of Supervised Learning Model
print("Supervised log.reg. score:", basemodel.score(X, ytrue))
```

Step 6: Self-Learning Model

Self-learning means:

- Train the model on labeled data.
- Predict labels for unlabeled data.
- Use confident predictions as new labeled data.

```
from frameworks.SelfLearning import SelfLearningModel

# Self-Learning Model
ssmodel = SelfLearningModel(basemodel)
ssmodel.fit(X, ys)

# Accuracy of Self-Learning Model
print("Self-learning log.reg. score:", ssmodel.score(X, ytrue))
```

Step 7: CPLE Model

In CPLE, the model:

- Uses **pessimistic constraints** to avoid trusting pseudo-labels too much.
- Only assigns labels if the model is **very confident**.

```
from frameworks.CPLELearning import CPLELearningModel

# CPLE Semi-Supervised Model
ssmodel = CPLELearningModel(basemodel)
ssmodel.fit(X, ys)

# Accuracy of CPLE Model
print("CPLE semi-supervised log.reg. score:", ssmodel.score(X, ytrue))
```

Final Output Example:

```
Supervised log.reg. score: 60%
Self-learning log.reg. score: 75%
CPLE semi-supervised log.reg. score: 85%
```

Advantages of CPLE

1. **Better Generalization**: CPLE can outperform both supervised and semi-supervised models, especially when the data labeling is expensive and the size of labeled data is small.
2. **Reduced Risk of Overfitting**: By being pessimistic and conservative about the predictions on unlabeled data, CPLE ensures that the model doesn't rely too heavily on potentially inaccurate predictions.
3. **Adaptability**: CPLE is particularly useful in **challenging domains** (e.g., rare disease detection, scientific research) where the **unlabeled data** may not be well represented in the labeled data, and gathering labeled data is resource-intensive.

Text Feature Engineering

1. Introduction to Text Feature Engineering

- Feature engineering is the process of **transforming raw data into meaningful features** for machine learning models.
- Traditional feature extraction methods work well for **numerical and categorical data**, but text data requires special handling.
- **Challenges with text data:**
  - Text is **unstructured** (does not have a fixed format).
  - Contains **irregularities** (misspellings, abbreviations, slang).

- **Difficult to convert into nume**
-
-
-
-
- **rical format** for machine learning models.

- **To handle text data, we use two main techniques:**

1. **Cleaning techniques** – Removing unnecessary and unwanted elements from text.
2. **Feature preparation techniques** – Transforming cleaned text into useful numerical representations.

---

2. Cleaning Text Data

- Text data in real-world applications is **rarely clean**.
- **Common problems in raw text datasets:**
  - **Misspellings** – Words may be misspelled, making them hard to process.
  - **Non-dictionary words** – Emoticons, abbreviations, and internet slang.
  - **HTML tags** – Data scraped from websites may contain <html>, , and other tags.
  - **Swear words disguised** using spaces/punctuation to bypass filters (e.g., f r e e instead of free).
  - **Repeated letters** – Some words may have extended letters (e.g., "heellloooo").
  - **Non-ASCII characters** – Symbols, emojis, and special characters that are not standard text.
  - **Hyperlinks** – URLs in comments or posts.

Example: The Impermium Dataset

- **2012 Kaggle Competition** dataset.
- **Goal:** Detect **insults** in social media comments using machine learning.
- **Example raw data:**

ID	Date	Comment
132	20120531031917Z	"""\xa0@Flip\xa0how are you not ded"""

  - The comment contains **unwanted escape characters (\xa0)** and **formatting issues**.

---

3. Text Cleaning with BeautifulSoup

- **Why Clean Text?**
  - **Better Model Performance** – Noisy text affects machine learning accuracy.
  - **Consistency** – Unstructured text makes analysis difficult.
  - **Faster Processing** – Large datasets with unclean text slow down computation.
- **Step 1: Inspect the Raw Text Data**
  - Before cleaning, manually check sample data.
  - Helps in **identifying common issues** (HTML tags, special characters, misspellings).
- **Step 2: Use BeautifulSoup for Initial Cleaning**
  - **What is BeautifulSoup?**
    - A **Python library** for processing and **cleaning HTML content**.
    - Extracts clean text by **removing HTML tags and escape characters**.
    - Works well on datasets containing **web-scraped or social media text**.

Python Code Example: Cleaning Text with BeautifulSoup

```
from bs4 import BeautifulSoup
import csv

trolls = [ ]  # List to store cleaned comments

with open('trolls.csv', 'rt') as f:
   reader = csv.DictReader(f)
   for line in reader:
```

```
        trolls.append(BeautifulSoup(str(line["Comment"]), "html.parser"))  # Clean HTML tags

        # Print first cleaned comment
        print(trolls[0])

# Extract plain text without HTML tags
eg = BeautifulSoup(str(trolls), "html.parser")
print(eg.get_text())
```

Expected Output (After Cleaning)

ID Date Comments
132 20120531031917Z @Flip how are you not ded

- **Before Cleaning:** """\xa0@Flip\xa0how are you not ded"""
- **After Cleaning:** @Flip how are you not ded (Escape characters removed).

Managing Punctuation and Tokenizing

Introduction to Tokenization

- Tokenization is the process of breaking down text into smaller components called tokens. These tokens are often words but can also include punctuation marks, character sets (such as emoticons), or other symbols. Effective tokenization is crucial for text preprocessing in Natural Language Processing (NLP).
- Before performing tokenization, it is essential to clean the raw text. Many datasets contain HTML elements, unnecessary spaces, and problematic characters that must be removed or replaced to ensure better processing. The re module in Python is a powerful tool that enables text cleaning using regular expressions.

Replacing Email Addresses and URLs

- Email addresses in text can be replaced with a generic token (_EM) to maintain privacy and standardization. This is done using the following regular expression:

```
text = re.sub(r'[\w\-][\w\-\.]+@[\w\-][\w\-\.]+[a-zA-Z]{1,4}', '_EM', text)
```

or

```
text = re.sub(r"\b[A-Za-z0-9._%-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b", "_EM", text)
```

- Similarly, since URLs are not useful for text processing, they can be replaced with the _U token:

```
text = re.sub(r'\w+:\/\/\S+', r'_U', text)
```

- This helps maintain consistency and avoids unnecessary variations in text caused by different email formats and URL structures.

Removing Extra Spaces and Special Characters

- Extra spaces, newline characters, hyphens, and underscores can be problematic. These elements are removed to ensure uniform text formatting:

```
text = text.replace("", ' ')
text = text.replace('\', ' ')
text = text.replace('_', ' ')
text = text.replace('-', ' ')
text = text.replace('\n', ' ')
text = text.replace('\\n', ' ')
text = re.sub(' +',' ', text)
```

- This step reduces unwanted variations in text while preserving essential information.

Managing Punctuation for Tokenization

- Extended punctuation (e.g., "??????" or "!!!!!!") is often used for emphasis in informal text. These can be replaced with specific tokens:
  - _BQ (for long question marks)
  - _BX (for multiple exclamation marks)
  - _Q (for single question marks)

    - _X (for single exclamation marks)
    - _SS (for multiple periods)
- Example regular expressions to replace them:

```
text = re.sub(r'([^!\?])(\?{2,})(\Z|[^!\?])', r'\1 _BQ\n\3', text)
text = re.sub(r'([^!\?])(\?|!){2,}(\Z|[^!\?])', r'\1 _BX\n\3', text)
text = re.sub(r'([^!\?])\?(\Z|[^!\?])', r'\1 _Q\n\2', text)
text = re.sub(r'([^!\?])!(\Z|[^!\?])', r'\1 _X\n\2', text)
```

- This step standardizes punctuation usage and prevents redundant characters from affecting text analysis.

Handling Repeated Characters and Swear Words

- In informal text, users often extend words for emphasis (e.g., "soooo happy"). These can be reduced to two consecutive characters and replaced with _EL:

```
text = re.sub(r'([a-zA-Z])\1\1+(\w*)', r'\1\1\2 _EL', text)
```

- Similarly, swear words or inappropriate symbols can be replaced with a general _SW token to avoid offensive content:

```
text = re.sub(r'([#%&\*\$]{2,})(\w*)', r'\1\2 _SW', text)
```

- This step ensures that offensive words and unnecessary word elongations do not influence text analysis.

Handling Smileys and Emoticons

- Emoticons are categorized into four groups and replaced with specific tokens:
    - _BS (big and happy smileys)
    - _S (small and happy smileys)
    - _BF (big and sad smileys)
    - _F (small and sad smileys)
- Example regular expressions to tokenize smileys:

```
text = re.sub(r' [8x;:=]-?(?:\)|\}|\]|>){2,}', r' _BS', text)
text = re.sub(r' (?:[;:=]-?[\)\}\]d>])|(?:<3)', r' _S', text)
text = re.sub(r' [x:=]-?(?:\(|\[|\||\\|/|\{|<){2,}', r' _BF', text)
text = re.sub(r' [x:=]-?[\(\[\|\\/\{<]', r' _F', text)
```

- Since emojis are constantly evolving, this method provides a structured way to handle basic emoticons while allowing for future updates.

Removing Non-ASCII Characters

- Non-ASCII characters, including certain emojis, may not be useful for text processing. These characters can be removed using:

```
text = re.sub(r'[^a-zA-Z]', '', text)
```

- This ensures that text remains in a processable format without unwanted symbols.

Splitting Text into Phrases and Words

- After preprocessing, text is split into phrases and then tokenized into words using str.split and re.findall:

```
phrases = re.split(r'[;:\.()\n]', text)
phrases = [re.findall(r'[\w%\*&#]+', ph) for ph in phrases]
phrases = [ph for ph in phrases if ph]
words = []
for ph in phrases:
    words.extend(ph)
```

- This step prepares the text for further analysis, such as classification or sentiment analysis.

Handling Spaced Single-Letter Chains

- Some users type spaced single-letter words (e.g., "h e l l o") to avoid detection. These sequences can be reconstructed:

```
tmp = words
words = []
new_word = ''
for word in tmp:
```

```
        if len(word) == 1:
            new_word = new_word + word
        else:
            if new_word:
                words.append(new_word)
            new_word = ''
        words.append(word)
```

- This helps in restoring meaningful words from misleading text sequences.

Challenges in Text Cleaning

Despite thorough preprocessing, challenges remain:

1. **Misspelled Words:** Spelling errors still exist after cleaning and may require spell-checking tools.
2. **Stopwords Removal:** Some words (e.g., "are," "pm") may not provide meaningful information and need removal.
3. **Corpus-Specific Variations:** Short text samples may not contain enough useful terms, making classification difficult.

Final Example

Before preprocessing:

```
GALLUP DAILY\nMay 24-26, 2012 \u2013 Updates daily at 1 p.m. ET;
reflects one-day change\nNo updates Monday, May 28; next update will
be Tuesday, May 29.\nObama Approval48%-\nObama Disapproval45%-1
```

After preprocessing:

```
['GALLUP', 'DAILY', 'May', 'Updates', 'daily', 'pm', 'ET',
'reflects', 'one', 'day', 'change', 'No', 'updates', 'Monday',
'May', 'next', 'update', 'Tuesday', 'Obama', 'Approval', 'Obama',
'Disapproval', 'PRESIDENTIAL', 'ELECTION', 'Obama', 'Romney',
'day', 'rolling', 'average', 'It', 'seems', 'bump', 'Romney',
'got', 'president', 'game']
```

Tagging and Categorizing Words

Introduction to Parts of Speech (POS) Tagging

- The **English language consists of different word types**, such as **nouns, verbs, adverbs, and adjectives**.
- These word types are known as **parts of speech**.
- **Identifying a word's category** helps in processing it differently, allowing **our algorithm to utilize it effectively**.

Importance of POS Tagging

- **Tagging words** as **nouns, adjectives, or verbs** allows algorithms to process text more efficiently.
- **Stop words** (such as *a*, *the*, *of*) add little value and should be removed to **reduce noise and variance** in training.

Using NLTK for POS Tagging and Stop Word Removal

- We use **the Natural Language Toolkit (NLTK)** to perform text tagging.
- **NLTK provides tools** to:
  - **Identify and encode word classes** as categorical variables.
  - **Improve data quality** by filtering out unimportant words.
- The **full range of text tagging techniques** is vast, but we focus on:
  - **N-gram tagging**
  - **Backoff taggers**

Stop Word Removal Using NLTK

- Stop words often contribute **little to text analysis** and should be removed.
- **Removing stop words** is straightforward using NLTK.

Implementation:

```
import nltk
nltk.download()
```

```
from nltk.corpus import stopwords
words = [w for w in words if not w in stopwords.words("english")]
```

Tagging with NLTK

Introduction to Tagging

- **Tagging** is the process of identifying **parts of speech (POS)** for words in a text and applying corresponding tags.
- A simple approach involves using a **dictionary-based tagging**, similar to how stopwords are identified:

```
tagged = nltk.word_tokenize(words)
```

- However, natural language is complex, and the same word (e.g., *ferry*) can function as multiple parts of speech depending on the context.
- Correct tagging requires **contextual understanding**, considering the words surrounding a given token.

1. Sequential Tagging

- **Definition**: A **sequential tagging algorithm** processes input data from **left to right**, tagging each word in succession.
- The decision for each tag depends on:
  - The current token.
  - The preceding tokens.
  - The predicted tags of the preceding tokens.
- **n-gram taggers** are a common type of sequential tagger, which rely on **(n-1) preceding POS tags** and the current token.

1. Unigram Tagger

- A **unigram tagger** assigns a part-of-speech tag to each word based only on the word itself. This means it looks at a single word and assigns the most likely part-of-speech tag based on its occurrence in the training data.

How it works:

- It tags each word with the most frequent tag observed for that word in the training corpus.
- It does not consider the surrounding words or their tags.

Example:

- If the word "love" appeared most frequently as a verb in the training data, the unigram tagger will assign the tag **VB** (verb) to "love" when it appears in the text.

Advantages:

- Simple to implement and fast to run.
- Requires less computational resources.

Disadvantages:

- Doesn't take into account the context, so it can make mistakes in ambiguous situations (e.g., "bank" can be a financial institution or the side of a river, but the unigram model wouldn't know based on surrounding words).

2. Bigram Tagger

- A **bigram tagger** assigns a part-of-speech tag to a word based on the current word and the tag of the previous word. It considers the sequence of two words (the current word and the previous word) to make a prediction about the current word's tag.

How it works:

- It uses the **previous word's tag** along with the current word to decide what tag to assign to the current word.
- For example, it might learn from training data that the word "love" is likely to be a verb when it follows a pronoun ("I love").

Example:

- If the word "I" is tagged as **PRP** (pronoun) and the word "love" follows, the bigram tagger might learn that "love" is most likely a verb (**VBP**) when preceded by a pronoun.

Advantages:

- Takes some context into account, which makes it more accurate than a unigram tagger in many cases.

- Can handle ambiguities more effectively by considering the previous tag.

Disadvantages:

- Limited to looking at only one previous word, so it may still struggle in cases where more context is needed (e.g., for longer-range dependencies between words).

3. N-gram Tagger (General)

- An **n-gram tagger** is a generalization of the unigram and bigram taggers. It uses the **previous n-1 tags** to predict the current word's tag. In an n-gram model, **n** can be any integer greater than 1, such as 3 (trigrams), 4 (4-grams), etc.

How it works:

- It considers the last **n-1 tags** (not just the previous one) when tagging the current word.
- For example, a trigram tagger will look at the current word and the previous two tags to predict the tag for the current word.

Example (for a trigram tagger):

- If the previous two tags are **PRP** (pronoun) and **VBP** (verb), the trigram tagger might learn that the next word "programming" is most likely a noun (**NN**).

Advantages:

- Provides more context by looking at multiple previous tags.
- Can capture longer-range dependencies and more complex linguistic patterns.

Disadvantages:

- Requires much more training data, as it has to account for a larger number of possible sequences.
- More computationally expensive because the number of possible n-grams increases with n, leading to a larger parameter space.

Example:

Given the sentence **"I love programming in Python"**, here's how you can generate unigrams, bigrams, and trigrams:

- **Unigrams** (1-gram):
  - ("I"), ("love"), ("programming"), ("in"), ("Python")
- **Bigrams** (2-gram):
  - ("I", "love"), ("love", "programming"), ("programming", "in"), ("in", "Python")
- **Trigrams** (3-gram):
  - ("I", "love", "programming"), ("love", "programming", "in"), ("programming", "in", "Python")

---

2. Backoff Tagging

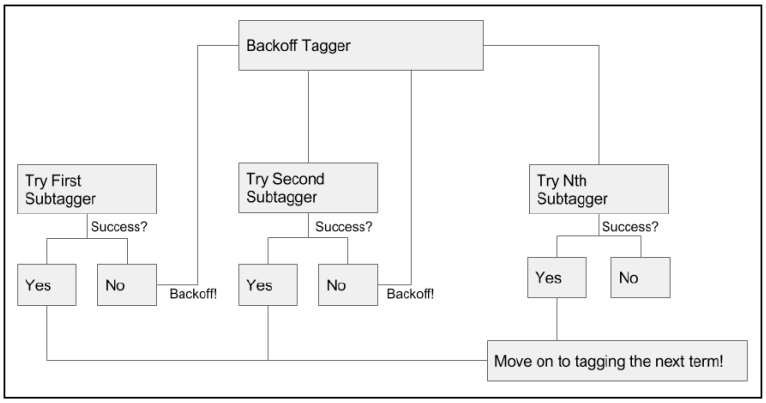
- **Tagging** assigns parts of speech (POS) to words based on their meaning and position in a sentence.
- Sometimes, a single tagger may **fail to accurately classify all tokens**, especially with **limited training data** or **uncommon words**.
- **Backoff Tagging** solves this problem by using **multiple taggers in combination**.

Concept of Backoff Tagging

- Backoff tagging is a type of **ensemble tagging technique** that uses multiple taggers.
- It works by **falling back** on simpler or more general taggers if the primary tagger fails.
- There are two types of taggers involved:
  1. **Subtaggers**: Primary taggers such as **n-gram taggers** or **Brill taggers**.
  2. **Backoff Taggers**: Secondary taggers that step in when the subtagger cannot classify a token.

How Backoff Tagging Works

1. The first tagger (**subtagger**) attempts to classify the token.
2. If the subtagger fails, the **backoff tagger** attempts to classify the token.
3. The process continues until a tag is assigned or all taggers are exhausted.
4. If no tagger assigns a tag, the token is assigned a **None** tag.



Example:

- let's implement a nested series of n-gram taggers. We'll start with a trigram tagger, which will use a bigram tagger as its backoff tagger.
- If neither of these taggers has a solution, we'll have a unigram tagger as an additional backoff. This can be done very simply, as follows:

```
import nltk
from nltk.tag import NgramTagger
from nltk.corpus import brown
nltk.download('brown')

# Load training data from Brown corpus (category 'a')
brown_a = nltk.corpus.brown.tagged_sents(categories='a')

# Initialize tagger with None
tagger = None
# Implement nested N-gram taggers with backoff
for n in range(1, 4):
    tagger = NgramTagger(n, brown_a, backoff=tagger)
# Example sentence
words = ["The", "quick", "brown", "fox", "jumps"]
print(tagger.tag(words))
```

Benefits of Backoff Tagging

- **Combines multiple taggers** for better accuracy.
- **Ensures redundancy**, reducing tagging failures.
- **Improves generalization**, handling both frequent and rare words.

Creating features from text data

- Once we have **cleaned** text data, we need to **convert** it into **useful features** for machine learning models. This process involves **feature engineering techniques** in **Natural Language Processing (NLP)**.
    1. Stemming
    2. Lemmatising
    3. Bagging using random forests

1. Stemming:

What is Stemming?

- Stemming is a **text normalization technique** used in Natural Language Processing (NLP) to reduce words to their base or root form.
- It involves **removing prefixes and suffixes** based on **predefined rules**, often leading to words that may not be **actual dictionary words** but still serve the purpose of reducing word variations.

Why is Stemming Important?

- Reduces **dimensionality** of text data
- Helps in **search engines** to find similar word forms (e.g., "running" and "run")
- Improves **text clustering** and **information retrieval**

How Stemming Works?

Stemming **simplifies words** by following **rule-based transformations**, such as:

1. **Removing suffixes**
   - "dances" → "danc"
   - "running" → "run"
   - "flies" → "fli"
2. **Applying heuristic rules**
   - "happiness" → "happi"
   - "beautifully" → "beautifuli"
3. **Handling double letters**
   - "controlling" → "control"
   - "strolling" → "stroll"
4. **Reducing multi-suffixes step-by-step**
   - "nationalism" → "nation"
   - "organization" → "organ"

Limitations of Stemming

- **May not return meaningful words** (*"flies" → "fli"*, *"happiness" → "happi"*)
- **Context is ignored** (e.g., *"better"* and *"good"* remain different)
- **Over-stemming problem** (e.g., *"universal" → "univers"*)
- **Under-stemming problem** (e.g., *"running"* and *"runner"* remain different)

Example: Using Porter Stemmer (NLTK)

```
from nltk.stem import PorterStemmer
# Initialize the Porter Stemmer
stemmer = PorterStemmer()
# Example words
words = ["running", "flies", "happiness", "dancing", "beautifully"]
# Apply stemming
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)
# Output: ['run', 'fli', 'happi', 'danc', 'beautifuli']
```

- Notice how **"flies" → "fli"**, **"happiness" → "happi"**, which are not actual words.
- This is the **limitation** of stemming. It does not always return meaningful words.

2. Lemmatization:

What is Lemmatization?

- Lemmatization is a **text normalization technique** in **Natural Language Processing (NLP)** that converts words to their **base dictionary form (lemma)** by considering **context and part of speech (POS).**
- Unlike **stemming**, which follows simple rules, **lemmatization uses a dictionary-based approach** to ensure words remain **valid and meaningful** after conversion.

Lemmatization vs Stemming?

Feature	**Stemming**	**Lemmatization**
Approach	Rule-based suffix removal	Dictionary-based analysis
Produces Valid Words?	✗ No (e.g., "flies" → "fli")	✓ Yes (e.g., "flies" → "fly")
Considers Word Meaning?	✗ No	✓ Yes
Handles Different POS?	✗ No	✓ Yes
Example	"running" → "run"	"running" → "run"
Example	"better" → "bet" ✗	"better" → "good" ✓
Computational Cost	✓ Faster	✗ Slower (requires POS tagging)

Why is Lemmatization Better?

- **Stemming:** "running" → "run", "flies" → "fli" (may not be a valid word).

- **Lemmatization:** "running" → "run", "flies" → "fly" (always a valid word).
- Also handles synonyms better (e.g., "books" and "tome" both → "book").

How Lemmatization Works

1. Identifies the **base form** of a word
2. Uses a **lexical dictionary (WordNet)**
3. Requires **POS tagging** for accuracy

Example: Using WordNet Lemmatizer (NLTK)

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
import nltk
nltk.download('wordnet')
# Initialize lemmatizer
lemmatizer = WordNetLemmatizer()
# Example words
words = ["running", "flies", "happiness", "dancing", "better"]
# Apply lemmatization with POS tagging
lemmatized_words = [lemmatizer.lemmatize(word, pos=wordnet.VERB) for word in words]
print(lemmatized_words)
# Output: ['run', 'fly', 'happiness', 'dance', 'better']
```

- **"flies" → "fly"**, **"dancing" → "dance"** (valid words).
- Works well for **nouns**, **verbs**, **adjectives**, and considers **POS tags**.

3. Bagging and Random Forests in Text Feature Engineering

- Bagging is a powerful ensemble learning technique that belongs to the family of **subspace methods**, which improve the stability and accuracy of machine learning models. Various subspace methods exist, each focusing on different aspects of data sampling:
  - **Pasting**: Involves drawing random subsets from the sample cases without replacement.
  - **Bagging (Bootstrap Aggregating)**: Involves sampling from cases **with replacement**, ensuring diversity among the models.
  - **Attribute Bagging**: Instead of drawing from cases, this method works with a **subset of features**, making it particularly useful for high-dimensional datasets such as those in medical and genetics applications.
  - **Random Patches Technique**: Combines both **sample cases and feature subsets** to build robust models.

Importance of Feature-Based Methods

- Feature-based techniques, such as **attribute bagging** and **random patches**, are crucial in handling high-dimensional datasets. In fields like **medical research** and **genetics**, where thousands of variables exist, these methods help in selecting the most relevant features while reducing computational complexity.

Bag of Words in NLP

- In **Natural Language Processing (NLP)**, **bagging** is commonly used in the **bag-of-words (BoW)** model. The BoW technique transforms text data into numerical form by counting word occurrences within a document. This representation allows machine learning models to analyze and classify text effectively.

Example of Bag-of-Words Representation

Consider the following dataset with two sample texts:

ID	Date	Words
132	20120531031917Z	['_F', 'how', 'are', 'you', 'not', 'ded']
69	20120531173030Z	['you', 'are', 'living', 'proof', 'that', 'bath', 'salts', 'effect', 'thinking']

- From these sentences, we extract the **unique words** (tokens), creating a **vocabulary list**:

["_F", "how", "are", "you", "not", "ded", "living", "proof", "that", "bath", "salts", "effect", "thinking"]

- Using this list as a reference, we can now **convert each sentence into a feature vector** by counting the occurrences of each word:

ID	Date	Comment	Bag-of-Words Vector
132	20120531031917Z	"_F how are you not ded"	**[1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]**
69	20120531173030Z	"you are living proof that bath salts effect thinking"	**[0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1]**

- Each vector represents a **numerical transformation** of the text data, making it suitable for machine learning algorithms.

Weighted Term Representations

Once the text data is transformed into numerical vectors, additional techniques can refine the representation:

1. **Binary Masking**: Converts word counts into binary values (1 if the word appears, 0 if it does not). This prevents **common words** from dominating the model.
    - Example: Instead of [3, 2, 0, 1], it would be [1, 1, 0, 1].
2. **Term Frequency-Inverse Document Frequency (TF-IDF)**:
    - This weighting scheme **increases** the importance of words that appear frequently within a specific document but are **rare** across the entire dataset.
    - TF-IDF is widely used in **search engines** and **text mining applications**.

Implementation of Bag-of-Words and TF-IDF in Python

Step 1: Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

Step 2: Initialize the Vectorizer

We create an instance of **TfidfVectorizer**, which converts text into numerical feature vectors:

```
vectorizer = TfidfVectorizer(analyzer="word",
                             tokenizer=None,
                             preprocessor=None,
                             stop_words=None,
                             max_features=5000)
```

- **analyzer="word"** → Analyzes the text at the word level.
- **tokenizer=None** → Uses default tokenization.
- **preprocessor=None** → No additional preprocessing is applied.
- **stop_words=None** → No stopwords are removed.
- **max_features=5000** → Limits the vocabulary size to **5,000 most important words**.

Step 3: Transform the Text Data

```
train_data_features = vectorizer.fit_transform(words)
train_data_features = train_data_features.toarray()
```

- **fit_transform(words)** → Converts the text into numerical features.
- **toarray()** → Converts the sparse matrix output into a standard array format.

Random Forests in Bag-of-Words Implementation

Once text data is converted into numerical vectors, it can be used in **machine learning models** for classification tasks. **Random forests** are particularly effective in handling high-dimensional text data because they:

1. **Reduce Overfitting**: Multiple decision trees vote on predictions, preventing reliance on noise.
2. **Handle High-Dimensional Data**: Feature bagging ensures that only the most relevant words are used.
3. **Work Well with Imbalanced Data**: Weighted voting helps address class imbalances.

Random forests operate as follows:

- Multiple **decision trees** are trained on different subsets of data.
- Each tree predicts a class, and the final output is determined by **majority voting**.
- In **boosting**, trees are iteratively added to improve model accuracy.

Due to their effectiveness, **random forests are often used as benchmark models** in text classification.

Applications of Bag-of-Words and Random Forests

1. **Spam Detection**: Detecting whether an email is spam or not.
2. **Sentiment Analysis**: Classifying text as **positive, negative, or neutral**.
3. **Insult Detection**: Identifying offensive language in online discussions.
4. **Search Engine Optimization (SEO)**: Using **TF-IDF** to rank web pages based on keyword relevance.
5. **Medical Text Analysis**: Extracting relevant features from **clinical documents** for disease prediction.

Example:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Sample dataset (Spam Detection)
texts = ["Win a free iPhone now", "Meeting at 5 PM", "Congratulations! You won a lottery", "Let's have lunch tomorrow"]
labels = [1, 0, 1, 0]  # 1: Spam, 0: Not Spam

# Bag-of-Words Representation
vectorizer = CountVectorizer()
X_bow = vectorizer.fit_transform(texts).toarray()
print(X_bow)

# TF-IDF Representation
tfidf_vectorizer = TfidfVectorizer()
X_tfidf = tfidf_vectorizer.fit_transform(texts).toarray()
print("Vocabulary:", vectorizer.get_feature_names_out())
print(X_tfidf)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_tfidf, labels, test_size=0.2, random_state=42)

# Train Random Forest Classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Predictions
y_pred = clf.predict(X_test)

# Evaluate Model
accuracy = accuracy_score(y_test, y_pred)
print(f"Spam Detection Model Accuracy: {accuracy * 100:.2f}%")
```

Testing our prepared data

- Now that we've done some initial preparation of the dataset, let's evaluate our model's performance. The dataset presents a single-class classification problem where the label is either 0 (neutral comment) or 1 (insulting comment). The predictions must be a real number in the range [0,1], with 1 indicating a high confidence that a comment is an insult.

Key Considerations:

- We are identifying comments that are insulting to participants in a blog/forum conversation.
- Insults directed at non-participants (e.g., celebrities) are not considered.
- Insults may contain offensive language but not necessarily so.
- Comments with profanity or slurs that are not insulting to a person are classified as non-insulting.
- The insulting nature should be obvious, not subtle.
- Label noise exists but is minimal (<1%).
- The problem has a tendency to overfit.

Initial Model: Random Forest Regressor

- A random forest is an ensemble of decision trees, balancing bias and variance through multiple trials. We begin by using 100 trees, which is a practical starting point for rapid iteration and learning about our dataset.

Model Training:

```
trollspotter = RandomForestRegressor(n_estimators=100, max_depth=10, max_features=1000)
y = trolls["y"]
trollspotted = trollspotter.fit(train_data_features, y)
```

Testing & Rescaling Predictions:

```
moretrolls = pd.read_csv('moretrolls.csv', header=True, names=['y', 'date', 'Comment', 'Usage'])
moretrolls["Words"] = moretrolls["Comment"].apply(cleaner)
y = moretrolls["y"]
test_data_features = vectorizer.fit_transform(moretrolls["Words"])
test_data_features = test_data_features.toarray()
pred = trollspotter.predict(test_data_features)
pred = (pred - pred.min())/(pred.max() - pred.min())
```

Evaluating AUC Score:

```
fpr, tpr, _ = roc_curve(y, pred)
roc_auc = auc(fpr, tpr)
print("Random Forest benchmark AUC score, 100 estimators")
print(roc_auc)
```

Results:

```
Random Forest benchmark AUC score, 100 estimators
0.537894912105
```

Improving Model Performance

To enhance performance, we adjust several parameters:

- Increase term cap from 5,000 to 8,000 terms.
- Increase the number of trees to 1,000.

Results:

```
Random Forest benchmark AUC score, 1000 estimators
0.546439310772
```

While an improvement, the score is still suboptimal. Next, we try an alternative classifier—Support Vector Machine (SVM).

Switching to Support Vector Machine (SVM)

```
class SVM(object):
    def __init__(self, texts, classes, nlpdict=None):
        self.svm = svm.LinearSVC(C=1000, class_weight='auto')
        if nlpdict:
            self.dictionary = nlpdict
        else:
            self.dictionary = NLPDict(texts=texts)
        self._train(texts, classes)

    def _train(self, texts, classes):
        vectors = self.dictionary.feature_vectors(texts)
        self.svm.fit(vectors, classes)
```

```
def classify(self, texts):
    vectors = self.dictionary.feature_vectors(texts)
    predictions = self.svm.decision_function(vectors)
    predictions = predictions / 2 + 0.5
    predictions[predictions > 1] = 1
    predictions[predictions < 0] = 0
    return predictions
```

Results:

SVM AUC score
0.625245653817

Next Steps for Improvement

1. **Feature Engineering Enhancements**
    - **Improve text preprocessing**:
        - Remove stopwords, stemming, lemmatization.
        - Experiment with **TF-IDF** instead of a simple **Bag of Words**.
    - Use **n-grams (bigrams, trigrams)** to capture phrase-based insults.
2. **Hyperparameter Tuning**
    - Optimize C in SVM (try values like **0.1, 1, 10, 1000**).
    - Tune Random Forest max_depth, n_estimators, min_samples_split.
3. **Try Word Embeddings (Advanced)**
    - Instead of **Bag of Words**, try **Word2Vec, FastText, or BERT embeddings**.
4. **Ensemble Methods**
    - Combine **Random Forest and SVM** in an **ensemble model**.
    - Use **stacking/blending techniques** to improve prediction accuracy.
5. **Error Analysis**
    - Check **which comments are being misclassified**.
    - Understand **patterns in false positives and false negatives**.
    - Adjust model parameters based on **misclassified cases**.